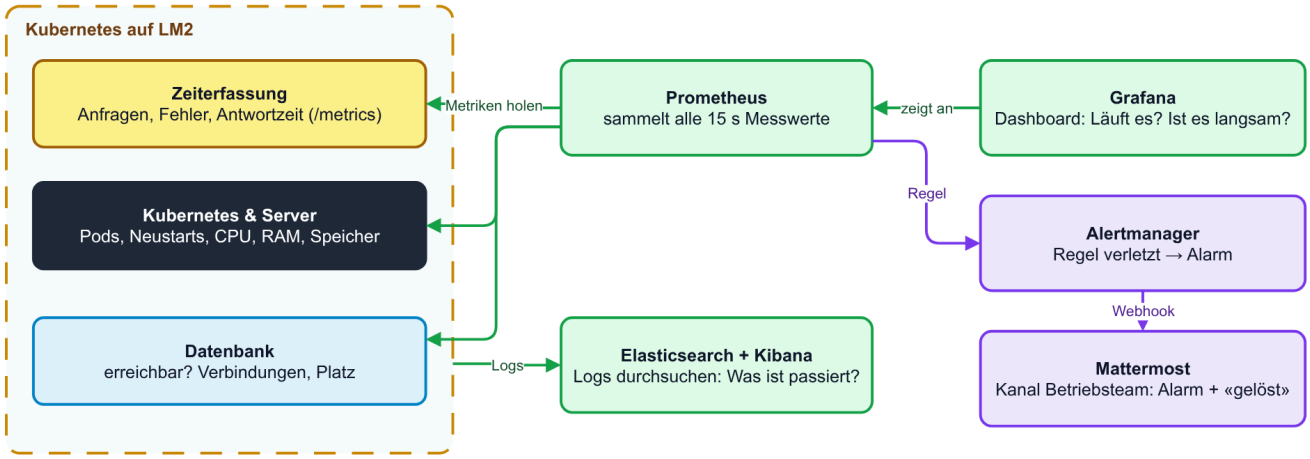
Monitoring für die Zeiterfassung

Damit Sie wissen, ob Ihre Zeiterfassung läuft, messen wir laufend, was die Anwendung und die Plattform darunter tun. Bei Problemen meldet sich das System von selbst beim Betriebsteam, bevor Ihre Mitarbeitenden es merken.

So überwachen wir die Zeiterfassung



Wie überwachen wir Kubernetes und Docker?

- **Prometheus** holt alle 15 Sekunden Messwerte ab (Pull-Prinzip): von der Zeiterfassung selbst (`/metrics`), vom Server (Node-Exporter: CPU, RAM, Disk), von den Containern (cAdvisor / Kubelet) und vom Cluster (kube-state-metrics: Pods, Neustarts, Deployments). Ohne Kubernetes, nur mit Docker, reichen cAdvisor und Node-Exporter.
- **Grafana** stellt diese Werte als Dashboard dar: auf einen Blick sehen, ob alles grün ist.
- **ELK**: Ein Log-Shipper (Filebeat) liest auf jedem Server die Logs der Container, **Elasticsearch** speichert sie, **Kibana** durchsucht sie. So beantworten wir «Was ist um 08:03 passiert?».
- **Alertmanager** prüft Regeln und schickt bei Verletzung eine Nachricht an **Mattermost**.

Was überwachen wir in der Zeiterfassung? (die vier goldenen Signale)

Signal	Was wir messen	Ziel
Latenz	Antwortzeit der Seite und beim Stempeln	95 % unter 0,5 s
Traffic	Anfragen und Stempelungen pro Minute	Verlauf, kein Grenzwert
Fehler	Anteil fehlgeschlagener Anfragen (HTTP 5xx), fehlgeschlagene Logins	unter 1 %
Sättigung	CPU und RAM gegenüber dem Limit, Datenbank-Verbindungen, freier Speicher	unter 80 %

Dazu die Verfügbarkeit: Health-Check der Anwendung, Pods bereit, Datenbank erreichbar.

Wann gibt es einen Alarm?

Zeiterfassung länger als 1 Minute nicht erreichbar · Fehlerquote über 5 % während 5 Minuten · Antwortzeit dauerhaft zu hoch · Pod startet immer wieder neu · Datenbank nicht erreichbar · Speicher über 85 % voll. Die Nachricht in Mattermost nennt Dienst, Zustand, Startzeit und einen Link zur Anleitung (Runbook). Ist das Problem behoben, folgt automatisch eine «gelöst»-Meldung.

Best Practices

- **Alarmieren, was Benutzer spüren**: lieber «Seite nicht erreichbar» als jede hohe CPU-Spitze.
- **Wenige, klare Alarme** mit Anleitung, sonst gewöhnt sich das Team an Fehlalarme und übersieht den echten.
- **Gleiche Bezeichnungen überall** (Dienst, Pod, Namespace): so passen Metriken und Logs zusammen.
- **Strukturierte Logs** (JSON), keine Passwörter oder Personendaten im Klartext.
- **Aufbewahrung begrenzen**: Metriken rund 15 Tage, Logs 7–30 Tage. Elasticsearch braucht am meisten Speicher.
- **Das Monitoring selbst überwachen**: kommen keine Daten mehr an, ist das auch ein Alarm.
- **Monitoring** meldet bekannte Fehlerbilder; **Observability** (Metriken, Logs, Traces verknüpft) hilft, die Ursache unbekannter Fehler zu finden.